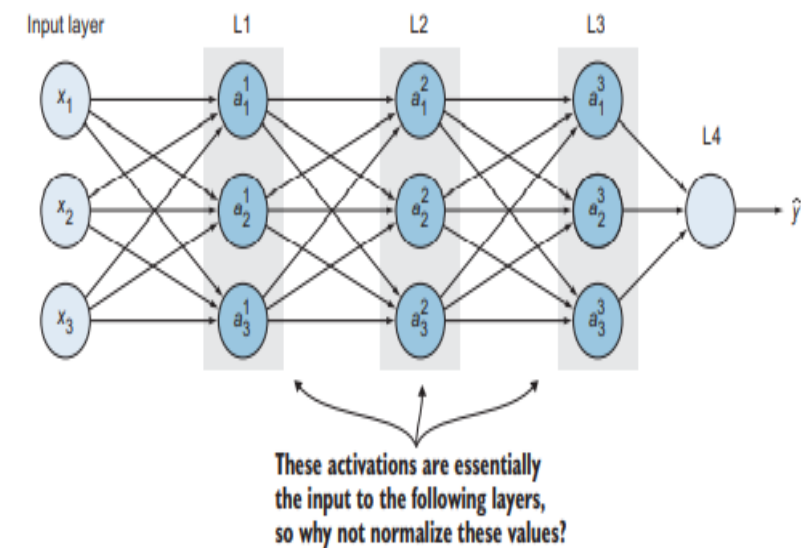


Deep Learning

Lec 8

Batch normalization

- We talked about data normalization to speed up learning. The normalization techniques we discussed were focused on preprocessing the training set before feeding it to the input layer.
- If the input layer benefits from normalization, why not do the same thing for the extracted features in the hidden units, which are changing all the time and get much more improvement in training speed and network resilience. This process is called **batch normalization**



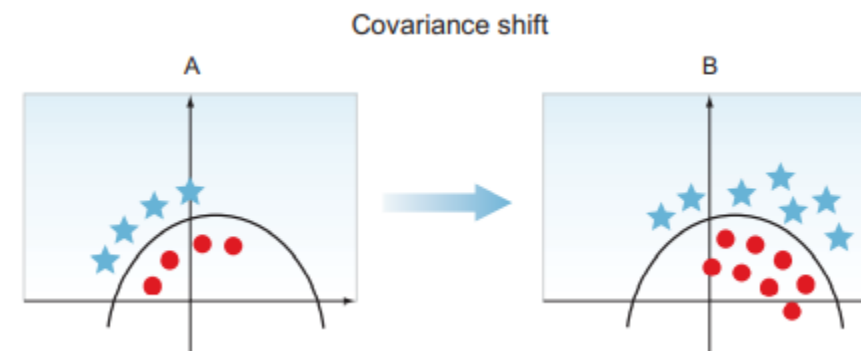
example to illustrate the problem that batch normalization (BN) confronts.

Suppose you are building a cat classifier, and you train your algorithm on images of white cats only.

When you test this classifier on images with cats that are different colors, it will not perform well. Why? Because the model has been trained on a training set with a specific distribution (white cats).

When the distribution changes in the test set, it confuses the model

We should not expect that the model trained on the data in **graph A** will do very well with the new distribution in **graph B**. The idea of the change in data distribution goes by the fancy name **covariate shift**.

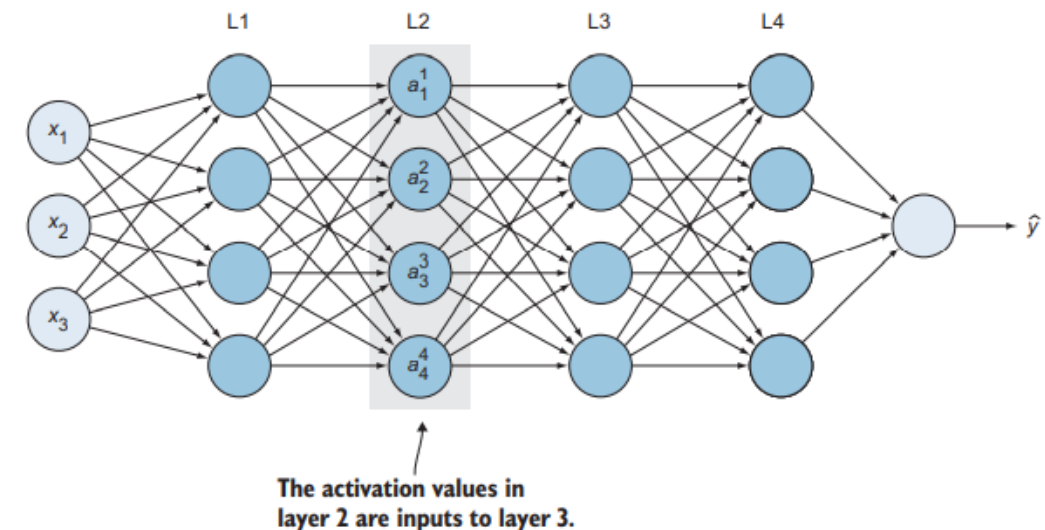


DEFINITION If a model is learning to map dataset X to label y , then if the distribution of X changes, it's known as covariate shift. When that happens, you might need to retrain your learning algorithm.

Covariate shift in neural networks

- To understand how covariate shift happens in neural networks, consider the simple four-layer MLP. Let's look at the network from the third-layer (L3) perspective.
- L3, its inputs are the activation values in L2 (a_1^2 , a_2^2 , a_3^2 , and a_4^2), which are the features extracted from the previous layers.
- L3 is trying to map these inputs to $\hat{y}$ to make it as close as possible to the label y . **While the third layer is doing that, the network is adapting the values of the parameters from previous layers.**
- As the parameters (w , b) are changing in layer 1, the activation values in the second layer are changing, too. So **from the perspective of the third hidden layer, the values of the second hidden layer are changing all the time:** the MLP is suffering from the problem of covariate shift.
- **Batch norm reduces the degree of change in the distribution of the hidden unit values, causing these values to become more stable so that the later layers of the neural network have firmer ground to stand on.**

It is important to realize that batch normalization does not cancel or reduce the change in the hidden unit values. What it does is ensure that the distribution of that change remains the same: even if the exact values of the units change, the mean and variance do not change.



- BN technique to reduce covariate shift. Batch normalization adds an operation in the neural network just **before the activation function** of each layer to do the following:
 - 1 . Zero-center the inputs
 - 2 . Normalize the zero-centered inputs
 - 3. Scale and shift the results .
- This operation lets the model learn the optimal scale and mean of the inputs for each layer.

How the math works in batch normalization

- 1 To zero-center the inputs, the algorithm needs to calculate the input mean and standard deviation (the input here means the current mini-batch: hence the term *batch normalization*):

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \longleftarrow \text{Mini-batch mean}$$

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad \longleftarrow \text{Mini-batch variance}$$

where m is the number of instances in the mini-batch, μ_B is the mean, and σ_B is the standard deviation over the current mini-batch.

- 2 Normalize the input:

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

where $\hat{x}$ is the zero-centered and normalized input. Note that there is a variable here that we added (ϵ). This is a tiny number (typically 10^{-5}) to avoid division by zero if σ is zero in some estimates.

- 3 Scale and shift the results. We multiply the normalized output by a variable γ to scale it and add (β) to shift it

$$y_i \leftarrow \gamma \hat{x}_i + \beta$$

where y_i is the output of the BN operation, scaled and shifted.

Notice that BN introduces two new learnable parameters to the network: γ and β . So our optimization algorithm will update the parameters of γ and β just like it updates weights and biases. In practice, this means you may find that training is rather slow at first, while GD is searching for the optimal scales and offsets for each layer, but it accelerates once it's found reasonably good values.

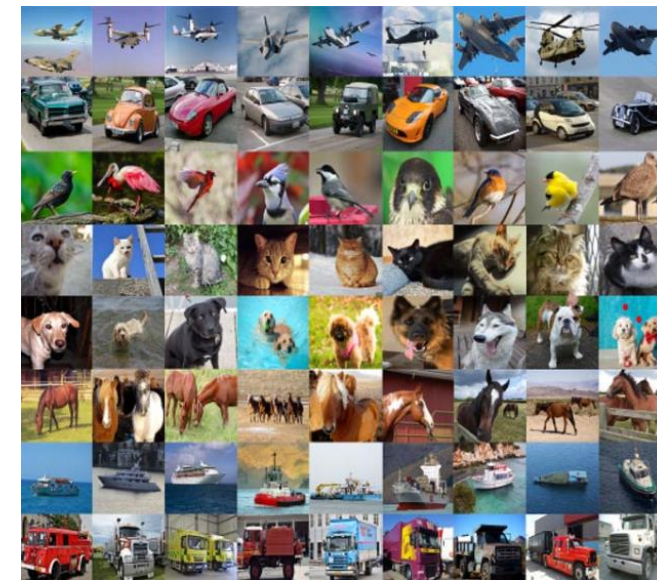
`model.add(BatchNormalization())` # Batch normalization layer have 4 trainable parameters

The project

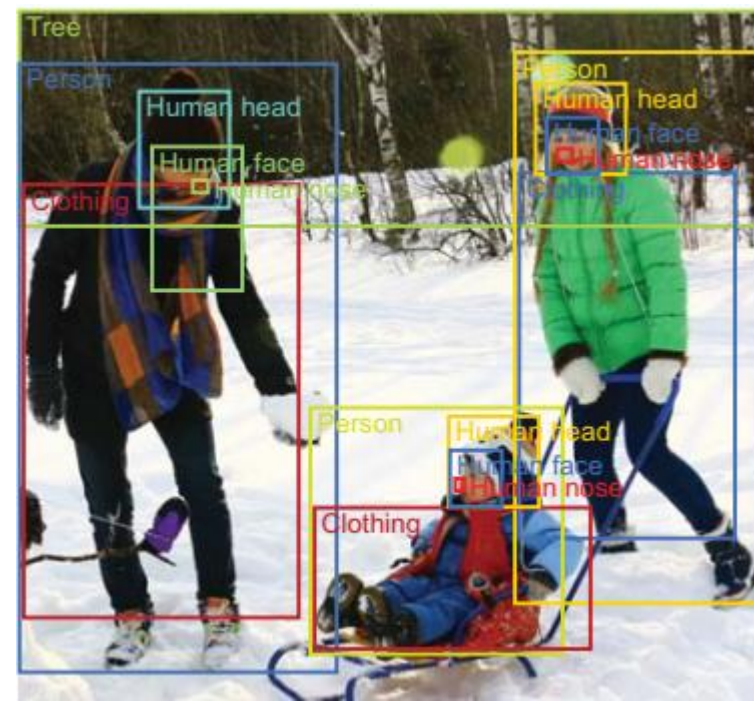
- We will accomplish the project by following these steps:
- 1 Import the dependencies.
- 2 Get the data ready for training:
 - – Download the data from the Keras library.
 - – Split it into train, validate, and test datasets.
 - – Normalize the data.
 - – One-hot encode the labels.
- 3 Build the model architecture. In addition to regular convolutional and pooling layers, as in chapter 3, we add the following layers to our architecture:
 - – Deeper neural network to increase learning capacity
 - – Dropout layers
 - – L2 regularization to our convolutional layers
 - – Batch normalization layers
- 4 Train the model.
- 5 Evaluate the model

Open source datasets

- **MNIST** : MNIST consists of 60,000 training images and 10,000 test images. All are grayscale (one-channel), and each image is 28 pixels high and 28 pixels wide. Figure shows some sample images from the MNIST dataset.
- **Fashion-MNIST**: it contains 60,000 training images and 10,000 test images of 10 fashion clothing classes: t-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, and ankle boot.
- **CIFAR** : The CIFAR-10 dataset consists of 32×32 color images in 10 classes, with 6,000 images per class. There are 50,000 training images and 10,000 test images.
- **ImageNet**: ImageNet is a large visual database designed for use in visual object recognition software research. It is aimed at labeling and categorizing images into almost 22,000 categories based on a defined set of words and phrases. The images were collected from the web and labeled by humans

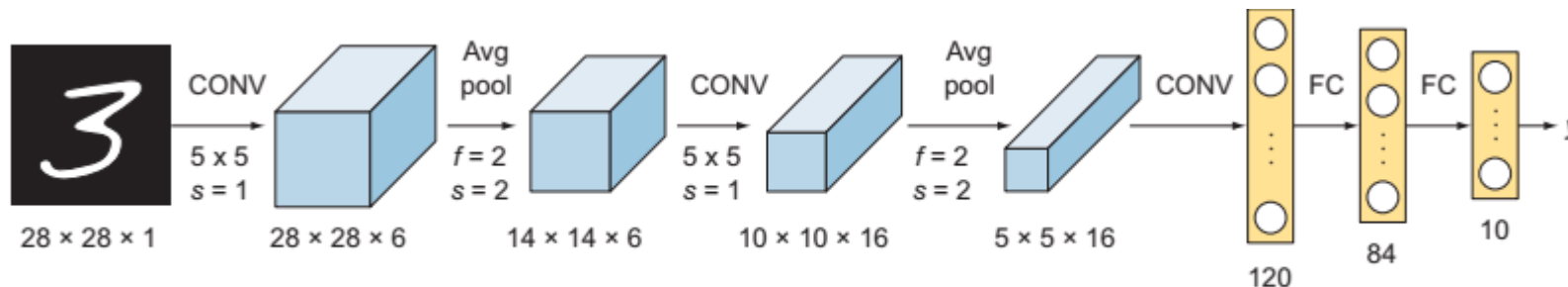


- **MS COCO:** Microsoft Common Objects in Context.: contains 328,000 images. More than 200,000 of them are labeled, and they include 1.5 million object instances and 80 object categories that would be easily recognizable by a 4-year-old.
- **Google Open Images:** It contains more than 9 million images as of this writing. What makes it stand out is that these images are mostly of complex scenes that span thousands of classes of objects. Additionally, more than 2 million of these images are hand-annotated with bounding boxes, making Open Images by far the largest existing dataset with object-location annotations . In this subset of images, there are ~15.4 million bounding boxes of 600 classes of objects.



CNN architectures

- some architectures that have implemented. Most of these architectures are famous because they performed well in the ImageNet competition.
- ImageNet is a famous benchmark that contains millions of images; DL and CV researchers use the ImageNet dataset to compare algorithms
- **LeNet-5** : In 1998, Lecun et al. introduced a pioneering CNN called *LeNet-5*. The LeNet-5 architecture is straightforward, and the components are not new to you (they were new back in 1998); you learned about convolutional, pooling, and fully connected layers. The architecture is composed of five weight layers, and hence the name LeNet-5: three convolutional layers and two fully connected layers.
- INPUT IMAGE $\Rightarrow$ C1 $\Rightarrow$ TANH $\Rightarrow$ S2 $\Rightarrow$ C3 $\Rightarrow$ TANH $\Rightarrow$ S4 $\Rightarrow$ C5 $\Rightarrow$ TANH $\Rightarrow$ FC6 $\Rightarrow$ SOFTMAX7
where C is a convolutional layer, S is a subsampling or pooling layer, and FC is a fully connected layer.



LeNet performance on the MNIST dataset

When you train LeNet-5 on the MNIST dataset, you will get above 99% accuracy

• AlexNet

The main motivation behind AlexNet was to build a deeper network that can learn more complex functions.

- AlexNet was the winner of the ILSVRC image classification competition in 2012. Krizhevsky et al. created the neural network architecture and trained it on 1.2 million high-resolution images into 1,000 different classes of the ImageNet dataset. AlexNet was state of the art at its time because it was the first real “deep” network that opened the door for the CV community to seriously consider convolutional networks in their applications.

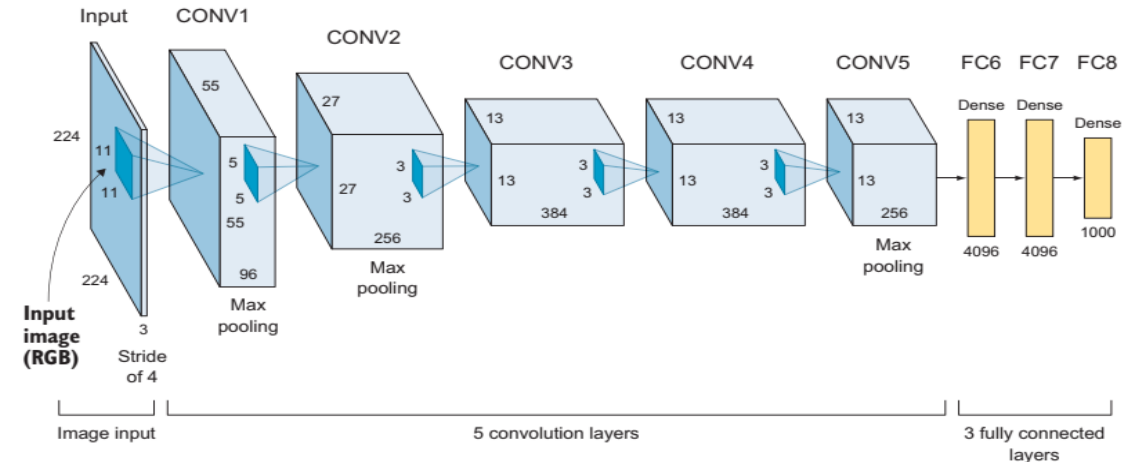
It consists of:

- Convolutional layers with the following kernel sizes: 11×11 , 5×5 , and 3×3
- Max pooling layers for images downsampling.
- Dropout layers to avoid overfitting

Unlike LeNet, ReLU activation functions in the hidden layers and a softmax activation in the output layer

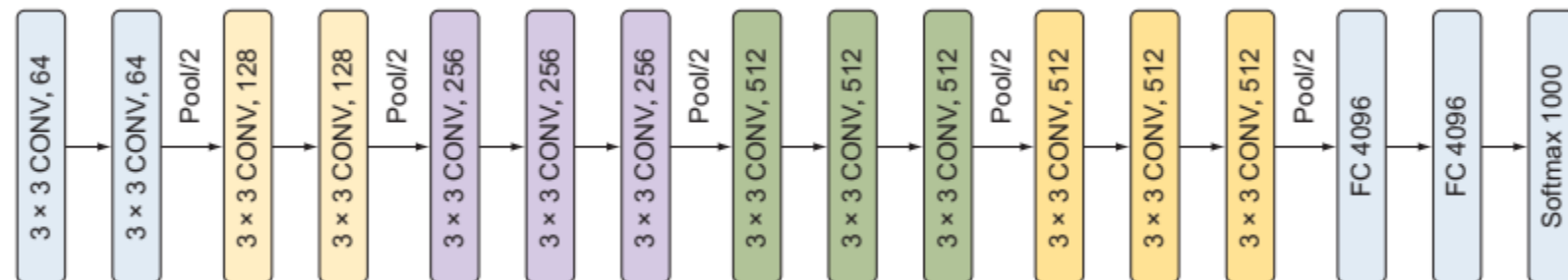
AlexNet consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully connected layers with a final 1000-way softmax. The architecture can be represented in text as follows:

INPUT IMAGE $\Rightarrow$ CONV1 $\Rightarrow$ POOL2 $\Rightarrow$ CONV3 $\Rightarrow$ POOL4 $\Rightarrow$ CONV5 $\Rightarrow$ CONV6 $\Rightarrow$ CONV7 $\Rightarrow$ POOL8 $\Rightarrow$ FC9 $\Rightarrow$ FC10 $\Rightarrow$ SOFTMAX



- *VGGNet*

- VGGNet was developed in 2014 by the Visual Geometry Group at Oxford University (hence the name VGG).³
- The building components are exactly the same as those in LeNet and AlexNet, except that VGGNet is an even deeper network with more convolutional, pooling, and dense layers.
- VGGNet, also known as VGG16, consists of 16 weight layers: 13 convolutional layers and 3 fully connected layers. Its uniform architecture makes it appealing in the DL community because it is very easy to understand.
- The architecture is composed of a series of uniform convolutional building blocks followed by a unified pooling layer, where:
 - All convolutional layers are 3×3 kernel-sized filters with a strides value of 1 and a padding value of same.
 - All pooling layers have a 2×2 pool size and a strides value of 2.
- use a smaller 3×3 kernel to allow the network to extract finer-level features of the image..
- VGG16 achieved a top-5 error rate of 8.1% on the ImageNet dataset compared to 15.3% achieved by AlexNet.



- *Inception and GoogLeNet*
- *ResNet*

later.....